COMP 462 / 561 - Homework #2
Due on October 19 2024, 23:59 on MyCourses

IMPORTANT: LATE SUBMISSIONS WILL NOT BE ACCEPTED, TO ENABLE THE RELEASE OF SOLUTIONS BEFORE THE MIDTERM EXAM.

Question 1 (30 points): Population genetics

For this question, we will simulate the evolution of a population of *N* diploid individuals (e.g humans), study how allele frequencies change over time and how this is impacted by selection.

a) (10 points) *For this question, submit your code as proof of work; however we will not execute your program.* Write a program that simulates the evolution of *N* diploid individuals with *L* SNPs. In this simulation, each individual is represented as a pair of binary vectors (0=reference allele, 1=alternate allele), corresponding to the alleles on the maternal and paternal chromosome. For example, if *L*=5, an individual could be represented as:
Maternal chromosome: 0 0 1 0 1
Paternal chromosome: 0 1 1 0 0
This would mean that this individual is homozygous for the reference allele at positions 0 and 3, heterozygous at positions 1 and 4, and homozygous for the alternate allele at position 2.

The simulation starts from some initial population (founding members), and then generates a next generation by repeatedly choosing two individuals as parents and producing an offspring from them. We will work with the following assumptions:

- Initial population:
  - The phased genotype information for the initial population is stored in the file initial_population.vcf . You can read more about the VCF file format here: https://samtools.github.io/hts-specs/VCFv4.2.pdf . However, no need to go in great details. Note that for each SNP, exactly one individual is heterozygous, while the others are homozygous for the reference allele. This means that the alternate allele frequencies of every SNP is 1/(2*N)*. This mimics an initial population with no standing variation, except for 100 mutations specific to each individual.

- Evolution:
  - We will simulate a *panmixing* population (i.e. a population where any two individuals can mate with any other; there are no males or females) of *constant* size *N*.
  - To go from a population at generation *G* to a population at generation *G*+1:

    For *i* in 0…*N*-1

Pick parent P_1 based on fitness wrt population Pop_G
Pick parent $P_2 \neq P_1$ based on fitness wrt population Pop_G
$Child_i$ = reproduce(P_1,P_2)
Pop_{G+1} = {$Child_0$, …, $Child_{N-1}$}

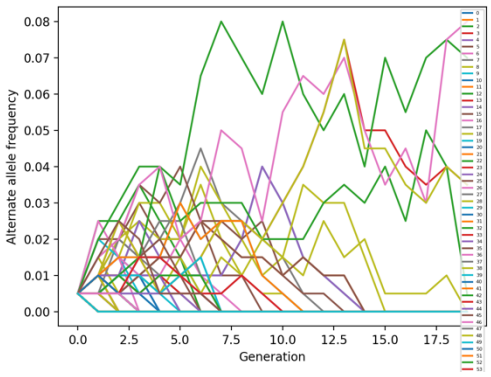
- Parent selection:
  - At each of the *N* iterations of the loop above, the probability that individual *i* is selected as parent P_1 is $fitness(i)/\sum_{j=1}^{N} fitness(j)$
  - The probability that individual $i' \neq P_1$ is selected as P_2 is

$$fitness(i') \Big/ \sum_{j=1\ldots N; j\neq P1} fitness(j)$$

    - Note: A given individual can be selected as parent any number of times. However, an individual cannot at the same time be the mother and the father of an offspring (i.e. the two parents need to be distinct).
- Reproduction: Suppose we have:
  Parent P_1 (mother) with maternal chrom $a_0,\ldots,a_{L-1}$ and paternal chrom $b_0,\ldots,b_{L-1}$
  Parent P_2 (father) with maternal chrom $c_0,\ldots,c_{L-1}$ and paternal chrom $d_0,\ldots,d_{L-1}$
    - Choose crossing-over position *k* uniformly among {0, 1, …, L-1}
    - Child maternal chromosome = $a_0\ldots a_k\ b_{k+1}\ldots b_{L-1}$ or $b_0\ldots b_k\ a_{k+1}\ldots a_{L-1}$ (with probability 0.5 each)
    - Choose crossing-over position *k'* uniformly among {0, 1, …, L-1}
    - Child paternal chromosome = $c_0\ldots c_{k'}\ d_{k'+1}\ldots d_{L-1}$ or $d_0\ldots d_{k'}\ c_{k'+1}\ldots c_{L-1}$ (with probability 0.5 each)
- Length of the simulation: Your code should be able to simulate the evolution of an initial population over any user-provided number of generations.

b) (5 points) Assume that all alleles are neutral, i.e. they do not impact fitness (fitness=1 for all individuals, irrespective of the alleles they carry). Calculate *analytically* the probability that a given alternate allele present at frequency 1/2*N* in generation 0 becomes *extinct* in generation 1. Express your solution as a formula that depends on *N* (the population size). Explain your work.

c) (2 points) Use your simulation code to estimate the probability of the event described in (b), still under the assumption that all alleles are selectively neutral. This is simply done by seeing what fraction of the 10,000 SNPs went to extinction in generation 1. Does it approximately match your result in (b)?



d) (2 points) Starting from the same initial population, simulate the evolution of a population over 20 generations, still assuming

that all alleles are selectively neutral. Plot the alternate allele frequency of the first 100 SNPs. The figure on the right is what I get; your result may differ, since this is a random process.

e) (5 points) Repeat the simulation, this time going for 1000 generations. Plot (i) the probability of allele extinction as a function of generation; (ii) the probability of allele fixation as a function of generation. These probabilities can be estimated by simply observing what fraction of the 10000 SNPs go to extinction or fixation.

f) (2 points) Now, assume that there is one specific SNPs (say SNP42) where being heterozygous confers a fitness of 1.5 (compared to a fitness of 1 for other alleles) and being homozygous for the alternate allele confers a fitness of 2. This means that an individual who is heterozygous at SNP42 is 50% more likely to be selected as a parent in the simulation, while an individual who is homozygous for the alternate allele at SNP42 is twice as likely to be selected. Calculate analytically the probability of extinction of the alternate allele at SNP42 after one generation. Use your simulation code to confirm your calculation, by running your 1-generation simulation repeatedly.

g) (2 points) Under the setting described in (f), use your simulation to estimate the probabilities of extinction and fixation of the alternate allele at SNP42, after 100 (not 1000) generations. To estimate these probabilities, you will need to run your simulation multiple times (I recommend 1000 times). Since the answer does not depend on the SNPs other than SNP 42, your simulation can involve only that SNP, which will make things faster.

h) (2 points) Now assume that the alternate allele at SNP42 is instead deleterious (it reduces fitness). Being heterozygous at SNP42 yields a fitness of 0.9, and being homozygous yields a fitness of 0.8. Use your simulation to estimate the probability of extinction and fixation after 100 (not 1000) generations. To estimate these probabilities, you will need to run your simulation multiple times (I recommend 1000 times). Since the answer does not depend on the SNPs other than SNP 42, your simulation can involve only that SNP, which will make things faster.

Question 2 (20 points): Genome-wide association study

In this question you will perform a small genome-wide association study on simulated data. The file gwas_population.vcf contains the genotypes of 1000 individuals at 10,000 SNPs. The file gwas_phenotypes.txt describes which of those 1000 individuals have disease D (a value of 1 indicates that the individual has the disease).

a) (10 points) Write a program that reads these two files and computes, for each SNP, the p-value of the association between the SNP's genotype and the disease D, using a standard chi-squared test. Suggestion: use Python's chi2_contingency function from

scipy.stats (or an analogous function). *Submit your code as proof of work; however we will not execute your program.*

b) (2 points) How many SNPs get an uncorrected p-value below 0.05? How does this compare to what you would expect by chance?

c) (5 points) List the SNPs that have a significant association (corrected p-value below 0.05) after correcting for multiple hypothesis testing using the Bonferroni correction. Give your answer in the form of a table of SNPs and corrected p-values. The corrected p-value for a given SNP is the product of the uncorrected p-value and the number of tests executed. Also provide an estimate of the disease odds ratio for individuals who are heterozygous and those who are homozygous for the alternate allele. The odds ratio is defined as:

Odds_ratio(het) = Pr [Disease | individual is het] / Pr [Disease | individual is homo-ref]
Odds_ratio(homo-alt) = Pr [Disease | individual is homo-alt] / Pr [Disease | individual is homo-ref]

Hint: Your table should contain a handful of results.

SNP ID	Uncorrected p-value	Corrected p-value	Disease odds ratio for heterozygous individuals	Disease odds ratio for homozygous alternate individuals

d) (3 points) Among the results you obtained in (c), some of SNPs obtain similar disease odds ratios but very different p-values. What may be causing this?

Question 3 (10 points): Genome Assembly

We have introduced in class an approach for *de novo* genome assembly, called shotgun sequencing. Shotgun sequencing works very well if the genome we aim to assemble is similar to a random DNA sequence. This is because random sequences are extremely unlikely to contain long repeated sequences. However, actual genomes typically look very different from random sequences. In particular, they often contain repetitive sequences, which are stretches of sequences (typically 100-10,000 bp in length) that are present at many locations in the genome. In the human genome, these repetitive sequences are often caused by the insertions of transposable elements.

Suppose that a mini genome sequence *G* is sequenced using shotgun sequencing, producing short reads of 5 bp. Assume that every index in *G* is the start position of a read (except the last

4, which are too close to the end of G). Give an example of a sequence *G* that would produce a set of reads for which both *G* and some other sequence G' would be equally good (i.e. short) genome assemblies.

Question 4. (20 points)
To answer this question, you will need to use some online tools implementing some of the algorithms seen in class. These might include:
Blast (http://www.ncbi.nlm.nih.gov/BLAST)
and
the LIRMM Phylogenetic inference package http://phylogeny.lirmm.fr/phylo_cgi/index.cgi

If you have never used Blast before, you may find this tutorial useful: http://www.ncbi.nlm.nih.gov/books/NBK1734/ . In particular, learn what an E-value is, and what the different Blast versions do (Blastn vs MegaBlast vs Blastp vs TBlastN, etc.)

Context: You are doctor and you have a patient suffering from a mysterious infection. You've extracted DNA from the infected area, sequenced it, and obtained the DNA sequence at http://www.cs.mcgill.ca/~blanchem/561/mystery.fa .

a) (10 points) What do you think is the cause of the infection? Hint: Default parameters for blastn may not result in the identification of very useful hits. Consider changing some of these parameters in order to try to identify hits with good E-values.

b) (10 points) Suppose there exist treatments for various strains of the pathogen. Five known strains exist, with sequences given at:
http://www.cs.mcgill.ca/~blanchem/561/strains.fa
Which treatment (i.e. that for which strain) is the most likely to be appropriate for your patient? Use a phylogenetic inference tool such as
http://phylogeny.lirmm.fr/phylo_cgi/index.cgi to figure it out. Explain your answer.

Question 5. (20 points)
Phylogenetic trees can be built from non-genetic data, and in fact this was the only type of information available prior to the 1970's, when DNA sequence became possible. Here, you will design and implement an algorithm similar to Sankoff's algorithm, but that will work on quantitative traits (things about a species that can be measured with integers, e.g. number of teeth) rather than genetic data. Suppose that you have information about *k* traits for each species. These traits are measured as non-negative integers. For example, for mammals, trait1 might be the length of the forearm (in cm), trait2 might the volume of the skull (in cm^3), trait3 might be the lifespan (in years), etc. As species evolve, their traits change but those changes are generally slow (although there are exceptions). Thus, a parsimony-based phylogenetic inference approach makes sense. The problem we want to solve is a version of the Small Parsimony Problem:

Given:

- A set of *n* species, with, for each species *i*, a vector of *k* integers $D_i = (D_{i,1}, D_{i,2}, \ldots, D_{i,k})$ representing the measurements made for the *k* traits in species *i*
- A phylogenetic tree *T* with leaves labeled with the *n* species.

Goal:
Assign a vector D_u to each internal node *u* of *T* such that $\sum_{(u,v)\in E(T)} d(D_u, D_u)$ is minimized, where $d(D_u,D_v) = \sum_{t=1}^{k} |D_{u,t} - D_{v,t}|$.

In other words, we want to make predictions about the vector of trait values present at each ancestral node, based on a parsimony principle.

a) (15 points) Write the pseudocode of an algorithm to solve this problem. You can assume that no trait value will ever exceed a given maximum value *M*. Note that our goal is to recover an assignment of ancestral trait vectors (not just the "parsimony score"), so if you use a dynamic algorithm similar to Sankoff's algorithm, make sure to explicitly write the traceback portion of the algorithm!

a) (5 points) What is the running time of your algorithm (using big-O notation)?